

Elastic *Stack*

Bilaketa, behagarritasuna eta log-en kudeaketa

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Sistemak



Elastic Stack

-
- 01 Sarrera: log kaosa eta pipeline-a
 - 02 Elasticsearch: alderantzizko indizea eta datu-eredua
 - 03 Query DSL: kontsultak eta agregazioak
 - 04 Kibana: bistaratzea eta ILM
 - 05 Ingesta: Beats eta Logstash (grok)
 - 06 Ariketak eta ondorioak

01 Sarrera eta arkitektura

“

Gaueko 3:00etan bezero batek deitu du: webgunea erori da. 50 zerbitzari daude. SSH, grep, SSH, grep... ordu bat igaro da. Oraindik ez dakigu non da arazoa.

— ELASTIC STACK-EN ZERGATIA · LOG KAOSA

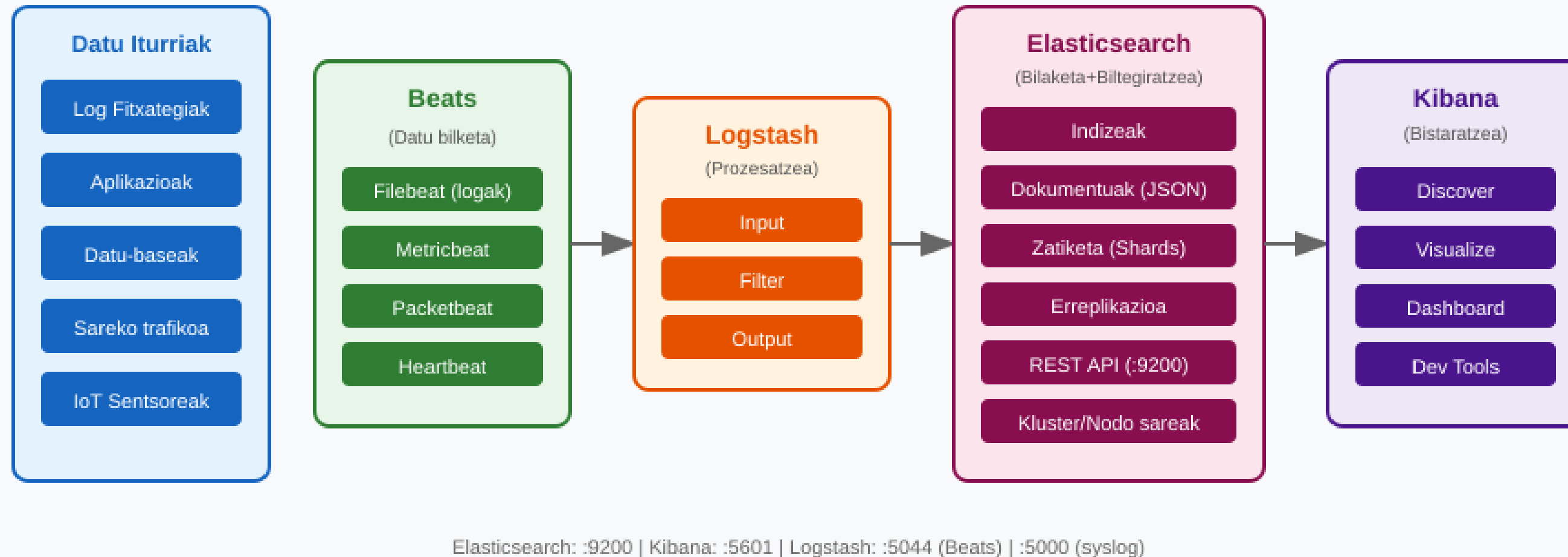
Zer da Elastic Stack?

- Kode irekiko plataforma datu kopuru handiak **biltzeko, bilatzeko, aztertzeko eta bistaratzeko**
- Historikoki **ELK Stack: Elasticsearch + Logstash + Kibana**
- Beats gehitu ondoren, **Elastic Stack** izena hartu zuen
- Erabilera nagusiak: **bilaketa, behagarritasuna, segurtasuna, IoT eta IA**
- Ez da soilik log-ak kudeatzeko sistema bat

— ARKITEKTURA

Elastic Stack: Datu-Fluxua

Iturria → Beats → [Logstash] → Elasticsearch → Kibana. Logstash aukerakoa da (datuak egituratuta badaude, zuzenean ES-era).



Hiru bloke nagusi

- **Observabilitatea** — sistemetan zer gertatzen ari den ikustea
logak, metrikak eta aplikazioen errendimendua
- **Segurtasuna** — Segurtasun-gertaerak aztertzea edo portaera susmagarriak eta iruzurra detektatzea
- **Bilaketa eta datuen erabilera** — adibidez, e-commerce batean milioika produkturen artean erabiltzaileak nahi duena oso azkar aurkitzea

Elastic Stack: Erabilera-kasuak

A. Observabilitatea

4 erabilera-kasu

-  Log Analisi Zentralizatua
-  APM — Errendimendu Monitorizazioa
-  Sistema Monitorizazioa (CPU/RAM)
-  LLM / AI Observability

B. Segurtasuna

3 erabilera-kasu

-  SIEM — Segurtasun Analisia
-  Iruzur Detekzioa
-  Zibersegurtasun Analitika

C. Datu Gestio & AI

5 erabilera-kasu

-  E-commerce Bilaketa
-  Mantenu Prediktiboa (IoT)
-  Analisi Geospazialak
-  Osasungintza
-  RAG eta Vektore-bilaketa

4 Osagaiak · Rol Bakoitza

- **Elasticsearch** — Datuak gorde, indexatu eta bilatzeko motor banatua. Stack-aren **bihotza**. REST API: **:9200**
- **Logstash** — Datuak jaso, eraldatu eta bidaltzeko pipelinea: **INPUT → FILTER → OUTPUT**
- **Kibana** — Elasticsearch-eko datuak kontsultatu, aztertu eta bistaratzeko web interfazea. **:5601**
- **Beats** — Datu-iturburuetan instalatzen diren **agente arinak**, datuak jaso eta bidaltzeko.

— KONPARAKETA

Noiz aukeratu Elastic?

Elastic Stack vs Alternatiboa

	Elastic Stack	Splunk	Grafana+Loki	SQL DB
Log analisia	✓	✓	✓	⚠
Testu bilaketa	✓	✓	✓	✗
SIEM segurtasuna	✓	✓	⚠	✗
Mantenu predikt.	✓	⚠	⚠	✗
Geospazialak	✓	✓	⚠	✓
RAG / AI	✓	⚠	⚠ ⚠	✗
Kostua	● Kode irekia	● Oso garestia	● Doakoa	● Merkea
Erraztasuna	● Ertaina	● Erraza	● Erraza	● Erraza

- **Splunk** — Antzeko erabilera-esparru zabala: logak, behagarritasuna eta segurtasuna. **Komertziala**, hainbat lizentzia/preczo eredurekiin.
- **Grafana + Loki** — Logen bilketa, bilaketa eta bistaratzeko aukera indartsua; Loki-k batez ere metadata/labels indexatzen ditu eta kostu txikiagoa bilatzen du.
- **SQL DB** — Datu egituratuak eta **transakzioak (OLTP)** kudeatzeko aproposa; ez da Elastic-en ordeztu zuzena testu-bilaketa masiborako.
- **Elastic Stack** — Bilaketa + log analisia + behagarritasuna + segurtasuna + bilaketa bektoriala/AI plataforma berean.

— 1. PRAKTIKA

ELASTICSEARCH + KIBANA MARTXAN JARRI

HELBURUA:

Docker Compose erabiliz **Elasticsearch** eta **Kibana** abiarazi eta bi zerbitzuak erabilgarri daudela egiaztatu.

PAUSOAK:

1. Sortu lanerako karpeta, **compose.yaml** fitxategiarekin
2. Ingurunea abiarazi
3. Probatu nabigatzailean:
 - Elasticsearch -> localhost:9200
 - Kibana → localhost:5601
4. Zer erakusten du Elasticsearch-ek localhost:9200 irekitzean?
Zer desberdintasun dago Kibana irekitzean?

```
compose.yaml
1  services:
2
3      elasticsearch:
4          image: docker.elastic.co/elasticsearch/elasticsearch:9.5.4
5          container_name: iabd-elasticsearch
6          environment:
7              - discovery.type=single-node
8              - xpack.security.enabled=false
9          ports:
10             - "9200:9200"
11          volumes:
12             - elasticsearch_data:/usr/share/elasticsearch/data
13
14      kibana:
15          image: docker.elastic.co/kibana/kibana:9.5.4
16          container_name: iabd-kibana
17          ports:
18             - "5601:5601"
19          environment:
20             - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
21          depends_on:
22             - elasticsearch
23
24  volumes:
25      elasticsearch_data:
```

02 Elasticsearch

Zergatik da Elasticsearch azkarra?

SQL-ren arazoa:

- Datu kopuru handietan **garestia** izan daiteke.
- Ez dago testu-bilaketarako bereziki optiizatuta.
- Ez du berez **errelevantzia-puntuaziorik**.
- **LIKE**-ek ez ditu berez antzeko hitzak edo akatsa ortografikoak kudeatzen.

Elasticsearch:

- Dokumentuak sartzean **alderantzizko indizea** eraikitzen du.
- Bilaketa **indize horren gainean** egiten du.
- Datuak **JSON dokumentu** gisa kudeatzen ditu.
- **REST API** bidez kontsultatzen da.
- **_score** → emaitzak errelevantziaren arabera ordenatzeko

GAKOA: ALDERANTZIZKO INDIZEA

Alderantzizko indizea

Liburu baten aurkibidea bezala: **hitza**
→ **zein dokumentutan** dagoen.
Bilaketa indizean egiten da, ez datuetan — millisegundotan.

Elasticsearch: Alderantzizko Indizea

Dokumentuak

Doc 1: "Elasticsearch bilaketa azkarra"
id=1

Doc 2: "Kibana bistaratzea azkarra"
id=2

Doc 3: "Logstash datuak bilaketa"
id=3



Alderantzizko Indizea

Hitza (Term)	Dokumentuak
bilaketa	Doc 1, Doc 3
azkarra	Doc 1, Doc 2
elasticsearch	Doc 1
kibana	Doc 2
bistaratzea	Doc 2
logstash	Doc 3
datuak	Doc 3

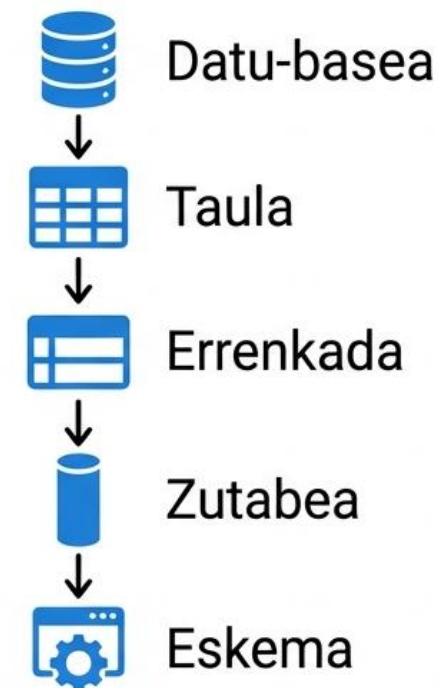
Bilaketa "bilaketa" → Doc 1 eta Doc 3 berehala aurkitu

Indizea, dokumentua, mapaketa

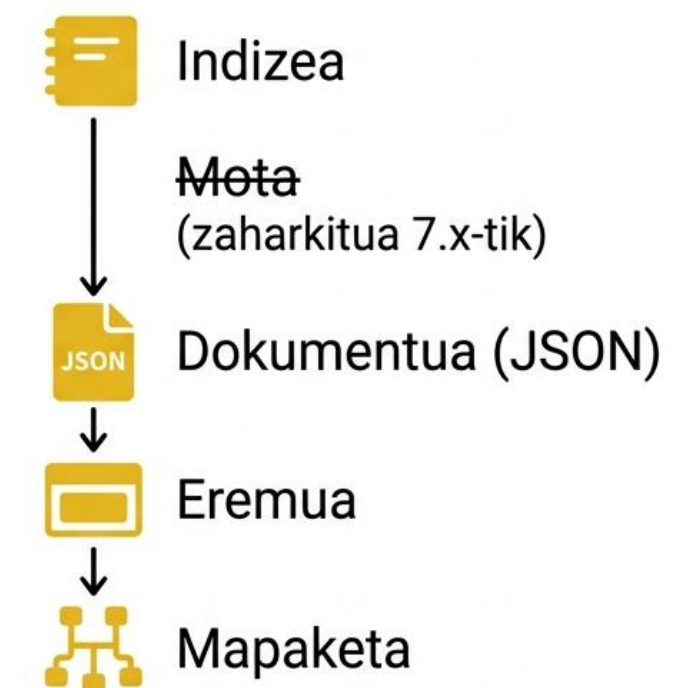
- Indizea ≈ taula
- Dokumentua ≈ errenkada → JSON
- Eremua ≈ zutabe
- Mapaketa (mapping) ≈ eskema
- ⚠ Ez da SQLko JOIN ereduan oinarritzen — desnormalizatu (dena dokumentu berean)
- Datu erlazionatuak dokumentu berean gordetzea → bilaketa sinpleago eta azkarragoa

Datu-eredua: Erlazionala vs Elasticsearch

Datu-base Erlazionala



Elasticsearch



⚠ ES-ek ez du JOIN operadorerik — harreman konplexuak aplikazio-mailan konpondu

Elasticsearch: Eremu Motak

⚡ **text** vs **keyword**: akats ohikoena!



text

Testu osoko bilaketa · Tokenizatua · match kontsultak · Ezin ordenatu
Koaderno gorria arina



keyword

Balio zehatza · Ez tokenizatua · term kontsultak · Ordenatu/Agregatu ✓
bulegoa



integer / long

Zenbaki osoak · range kontsultak
42



float / double

Zenbaki hamartarrak
2.99



date

Data eta orduak · ISO8601 formatua · now-1h espresioak
2024-01-15



boolean

Egia/Faltsua · Iragazketa sinplea
true / false



geo_point

Lat/Lon kokapena · Gertuko bilaketa · Geo-fencing
{lat: 43.26, lon: -2.92}



nested

Objektu-zerrenda · Harreman konplexuak
[{...}, {...}]

— EREMU-MOTAK

text vs keyword

text — tokenizatua, `match`
bilaketarako (testu librea).

keyword — bere horretan, `term`
bilaketarako (iragazi, ordenatu,
agregatu).

Kontzeptuen Baliokideak

DATU-BASE ERLAZIONALA	ELASTICSEARCH	OHARRA
Datu-basea	Indizea	Dokumentu-bilduma bat
Taula	Indizea	Oharra: mota zaharkitua dago 7.x-tik
Errenkada	Dokumentua	JSON objektua — oinarrizko unitatea
Zutabea	Eremua	JSON gako bat balioarekin
Eskema	Mapaketa	Eremu bakoitzaren mota definituta
JOIN	Ez dago	Aplikazio-mailan konpondu

Pitfall klasikoena

text vs keyword

text eremu batean term kontsulta egiteak EZ du ezer aurkitzen: text tokenizatuta eta txikiz gordetzen da ('koadernoa'), baina term-ek balio zehatza bilatzen du ('Koadernoa').
Konponbidea: izena.keyword.

GARRANTZITSUA

Askotan eremu bera bietara nahi da → multi-field mapaketa:
izena (text, bilatzeko) + izena.keyword (keyword, agregatzeko).

— 2. PRAKTIKA

LEHEN INDIZEA ETA DOKUMENTUAK

HELBURUA:

Kibana **Dev Tools** erabiliz Elasticsearch-en indize bat sortu, JSON dokumentuak txertatu eta datuak kontsultatu.

PAUSOAK:

1. Ireki Kibana → Dev Tools

2. Sortu lehen dokumentua:

```
1 POST produktuak/_doc
2 {
3   "izena": "Koaderno urdina",
4   "kategoria": "Papergintza",
5   "prezioa": 4.50,
6   "stock": 25
7 }
```

3. Gehitu beste 3 dokumentu:

```
POST produktuak/_doc
{
  "izena": "Koaderno handia",
  "kategoria": "Papergintza",
  "prezioa": 8.50,
  "stock": 10
}
```

```
POST produktuak/_doc
{
  "izena": "Sagu optikoa",
  "kategoria": "Informatika",
  "prezioa": 19.99,
  "stock": 15
}
```

```
POST produktuak/_doc
{
  "izena": "Teklatu mekanikoa",
  "kategoria": "Informatika",
  "prezioa": 59.99,
  "stock": 5
}
```

4. Kontsultatu indizeko dokumentuak:

```
GET produktuak/_search
```

5. Aztertu emaitza:

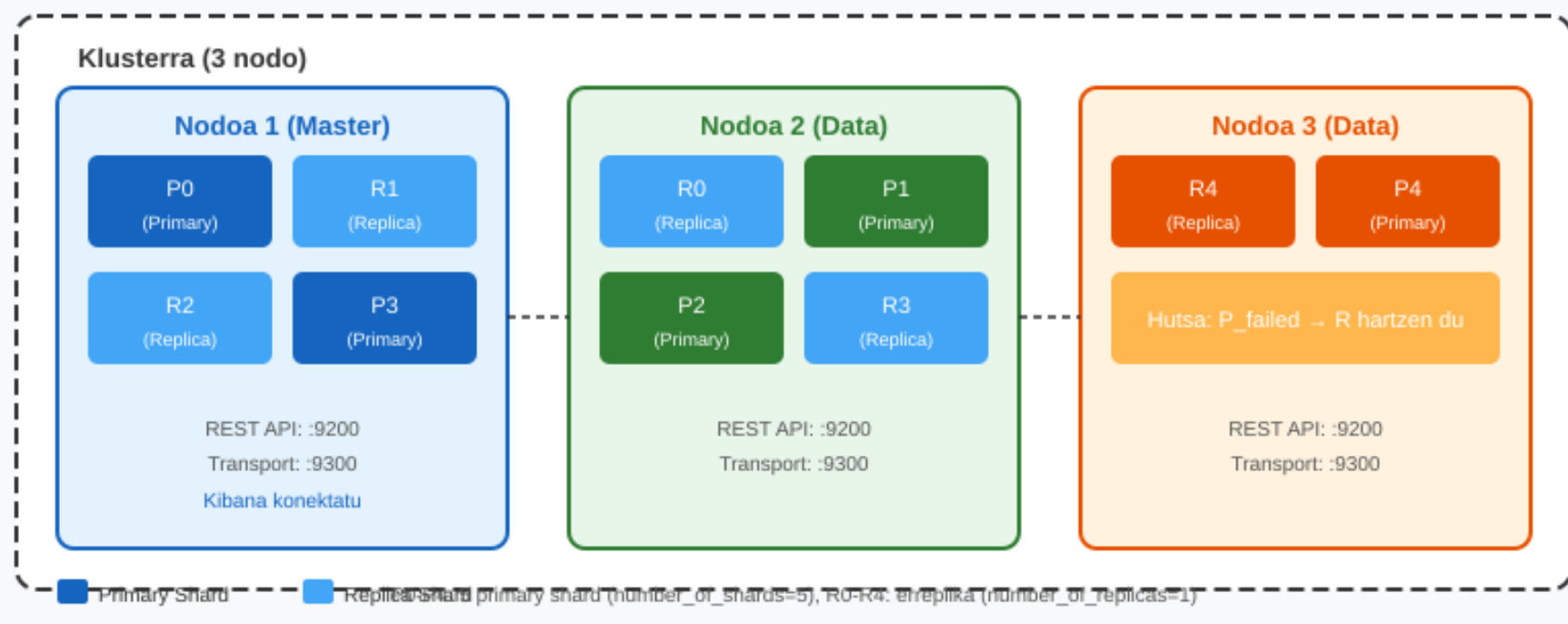
- Zenbat dokumentu daude?
- Non agertzen dira dokumentuaren benetako datuak?
- Zer da produktuak?
- Zer da { "izena": ..., "prezioa": ... } bakoitza?

— ESKALA

Shards eta replicas

- **Shard** — indizearen zati bat. Datuak nodoen artean **banatzeko** erabiltzen da → eskala horizontala
- **Replica** — shard baten **kopia**. Erabilgarritasuna eta bilaketa-errendimendua hobetzen ditu.
- ⚠ **Primary shard kopurua** indizea sortzean definitzen da; aldatzeko normalean **reindexatu** behar da. **Replica kopurua alda daiteke**.
- **Klusterraren osasuna:**
 - ● GREEN – primary eta replica guztiak esleituta
 - ● YELLOW – primary guztiak esleituta, baina replica batzuk ez
 - ● RED – primary shard bat edo gehiago ez dago erabilgarri

Elasticsearch: Zatiketa (Shards) eta Erreplikazioa



— 3. PRAKTIKA

CLUSTER HEALTH ETA MAPPING-A AZTERTU

HELBURUA: Elasticsearch klusterraren egoera aztertu eta **produktuak** indizearen **shard-ak eta mapping-a** ikusi.

PAUSOAK:

1. Klusterraren egoera aztertu.

```
GET _cluster/health
```

- Identifikatu: **status**, **number_of_nodes**, **active_primary_shards**, **active_shards**

2. **produktuak** indizearen shard-ak aztertu:

```
GET _cat/shards/produktuak?v
```

- Identifikatu: Primary shard-a (**p**), Replica (**r**), **STARTED** / **UNASSIGNED** egoerak

4. Indizearen mapping-a kontsultatu.

```
GET produktuak/_mapping
```

- Identifikatu Elasticsearch-ek automatikoki esleitutako datu-motak: izena, kategoria, prezioa, stock

5. Erantzun:

- a. Zein da klusterraren egoera? Zergatik?
- b. Zer gertatzen da replica-rekin? Zergatik?
- c. Zein datu-mota esleitu dio Elasticsearch-ek eremu bakoitzari?
- d. **izena** eta **kategoria** eremuetan, zergatik agertzen dira **text** eta **keyword**?

03

Query DSL eta agregazioak

Query DSL: Elasticsearch-en kontsultatzeko lengoaia

JSON formatuan idazten da → SQL-ko **WHERE**-en antzeko funtzioa.

Bi kontsulta-familia nagusi:

- **FULL-TEXT** → **text** eremuetarako
 - **match** → testu-bilaketa
 - **multi_match** → hainbat eremutan bilatu
 - **match_phrase** → esaldi/hurrenkera zehatza bilatu
 - Errelebantzia kalkulatzen du → **_score**
- **TERM-LEVEL** → balio zehatz, zenbaki eta datetarako
 - **term** / **terms** → balio zehatzak
 - **range** → balio-tarteak (> , < , datak...)

Full-text vs Term

Full-text (`match`) — text eremuetan, analizatua, `\_score`.

Term (`term`, `range`) — balio zehatza, ez tokenizatu.

Elasticsearch: Query DSL Kontsulta Motak

Full-text Kontsultak

text eremuetan · tokenizatua · `\_score` kalkulatu



match

Eremu batean hitz-bilaketa (OR/AND)



multi_match

Hainbat eremutan aldi berean (pisuekin)



match_phrase

Esaldi osoa, hurrenkera berean

Erabilera: testu libreko bilaketa

Term Kontsultak

keyword/zenbaki/data · zehatza · cacheatua



term

Balio zehatza



terms

Balio-zerrenda (IN)



range

Tartea: gte, lte, gt, lt



exists

Eremu existentzia



wildcard/fuzzy

Patroiak eta akats-tolerantzia

Erabilera: iragazketa zehatza

Konposatutako Kontsultak

beste kontsultak konbinatu



bool

must + should + must_not + filter

- **must**: beharrezkoa + `\_score`
- **should**: aukerakoa + `\_score` hobetu
- **must_not**: baztertu
- **filter**: beharrezkoa + cacheatua ⚡

Erabilera: kontsulta konplexuak

💡 **filter > must** errendimenduarentzat — ez du `\_score` kalkulatu eta cacheatzen da

Bool Kontsulta: Lau Klausulak

must ✓

Dokumentua **BETE BEHAR** du
_score-an **ERAGINA DU** ✓

```
{ "match": { "izena": "koaderno" } }
```

SQL: **"AND"**

should 🤔

🤔 Dokumentua bete **DEZAKE** _score **HOBETU** egiten du ?
minimum_should_match: 1

```
{ "term": { "kategoria": "bulegoa" } }
```

SQL: **"OR (puntuazioa hobetu)"**

bool

must_not ✗

Dokumentua **EZ** du bete behar ✗
Ez du _score-an eraginik ✗

```
{ "term": { "eskuragarri": false } }
```

SQL: **AND NOT**

filter ⚡

⚡ Dokumentua **BETE BEHAR** du ⚡
Ez du _score-an eraginik ⚡
CACHEATUA → AZKARRAGOA

```
{ "range": { "prezioa": { "gte": 1 } } }
```

SQL: **AND (optimizatua)**

⚡ **Arau praktikoa:** Zenbakizko iragazketak eta datak BETI filter barruan — must baino azkarragoa

— TRESNA NAGUSIA

Bool kontsulta – 4 klausula

Bool kontsulta hainbat kontsulta edo baldintza **konbinatzeko** erabiltzen da.

- **must** → bete behar da
- **should** → lehentasuna eman
- **must_not** → baztertu
- **filter** → iragazi, _score-an eragin gabe

must vs filter — errendimendua

	MUST	FILTER
Bete behar du	Bai	Bai
_score kalkulatu	Bai	EZ
Cacheatua	Ez	Bai ⚡
Noiz	Testu-bilaketa	Zenbaki, data, balio zehatza

— 4. PRAKTIKA

QUERY DSL: LEHEN BILAKETAK

HELBURUA: Query DSL erabiliz **produktuak** indizean **testu-bilaketak**, **prezio-iragazkiak** eta **baldintza konbinatuak** egitea

PAUSOAK:

1. Bilatu izenean **koaderno** hitza duten produktuak:

```
GET produktuak/_search
{
  "query": {
    "match": {
      "izena": "koaderno"
    }
  }
}
```

- Aztertu emaitzetako **_score** balioa

2. Bilatu 10 € edo gutxiago balio duten produktuak:

```
GET produktuak/_search
{
  "query": {
    "range": {
      "prezioa": {
        "lte": 10
      }
    }
  }
}
```

3. Konbinatu bi baldintzak bool erabiliz:

```
GET produktuak/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "izena": "koaderno"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "prezioa": {
              "gte": 1,
              "lte": 10
            }
          }
        }
      ]
    }
  }
}
```

— 4. PRAKTIKA

QUERY DSL: LEHEN BILAKETAK

HELBURUA: Query DSL erabiliz **produktuak** indizean **testu-bilaketak**, **prezio-iragazkiak** eta **baldintza konbinatuak** egitea

PAUSOAK:

4. Erantzun:

- a. Zein produktu aurkitzen ditu **match** kontsultak?
- b. Zein produktu aurkitzen ditu **range** kontsultak?
- c. Zein produktu geratzen dira azken **bool** kontsultan?
- d. Zer funtzio betetzen du **must**-ek? Eta **filter**-ek?

5. **Erronka:** Sortu kontsulta bat 10€ eta 50€ arteko produktuak bilatzeko.

ARIKETA GEHIGARRIAK

1. Bilatu **20€ baino garestiagoak** diren produktuak
2. Bilatu **izena** eremuan **mekanikoa** hitza duten produktuak
3. Bilatu **5€ eta 25€ arteko** produktuak.
4. **bool** erabiliz, bilatu izenean **koaderno** duten eta **5€ baino garestiagoak** diren produktuak.
5. **Erronka**: bilatu **Informatika** kategoriako produktuak **term** erabiliz. Zer gertatzen da **kategoria** erabiltzen baduzu? Eta **kategoria.keyword** erabiltzen baduzu?

Elasticsearch: Agregazio Motak



Bucket Agregazioak

Dokumentuak taldetan sailkatu

terms: kategoriaka taldekatu **date_histogram:** eguneko taldeak
range: prezio-tarteak
→ SQL GROUP BY baliokidea



Metrika Agregazioak

Talde barruko kalkulu numerikoak

avg: batezbestekoa **max:** maximoa **min:** minimoa
sum: batura **stats:** laburpen osoa **percentiles:** p50/p95/p99
→ SQL SUM(), AVG(), COUNT() baliokidea



Kabiatutako Agregazioak

Bucket barruan metrika kalkulatu

terms (kategoriak) → **avg** (batezbesteko prezioa)
→ **max** (prezio maximoa)
→ **value_count** (produktu kopurua)
→ "Kategoria bakoitzeko batezbesteko prezioa"

bulegoa:	45 prod	avg €3.20	■■■■■■■
tresnak:	23 prod	avg €8.50	■■■■■■■■■■■■■■■■■■■■
papeleria:	12 prod	avg €1.80	■■

— ANALITIKA

Agregazioak: datuak laburbildu eta analizatu

- **Bucket** → dokumentuak taldekatu
terms, date_histogram, range
- **Metrika** → kalkuluak egin
avg, sum, min, max, percentiles
- **Kabiatuak** → talde bakoitzaren barruan kalkuluak egin

04 Kibana

Kibana: Elasticsearch-en interfazea

Kibana Elasticsearch-eko datuak esploratu, bistaratzeak sortu eta administratzeko web-interfazea da.

Tresna nagusiak:

- **Discover** — dokumentuak esploratu eta iragazi
- **Lens** — grafikoak sortu
- **Dashboard** — bistaratzeak panel batean konbinatu
- **Dev Tools** — Query DSL bidez Elasticsearch kontsultatu
- **Stack Management** — indizeak eta ILM kudeatu

Portu lehenetsia: <http://localhost:5601>

IDEIA GAKOA: Kibana interfazea da; Elasticsearch da motorra.
Kibana-k Elasticsearch-era kontsultak bidaltzen ditu atzean.

Kibana: Tresna Nagusiak

Discover

Datu-esploratzea

Bilaketa adierazpenak (KQL)

Denboratarte filtratzea

Eremu aukeraketa

Dokumentu zuzenak ikusi

Visualize / Lens

Bistaratzek sortu

Bar/Line/Pie grafikak

Metrika eta taulak

Drag & drop interfazea

TSVB denb.-lerroak

Dashboard

Ikuspegi globala

Bistaratzek konbinatu

Interaktiboki iragaztea

Auto-freskatzea

Exportatu (PDF, PNG)

Maps

Mapa bistaratzea

Kokapena-oinarritutako analisia

geo_point eta geo_shape

Datu-geruza anitzak

Choropleth mapak

Dev Tools Console

GET /produktuak/_search

```
{ "query": { "match": {  
  "izena": "elasticsearch"  
} } }
```

Elasticsearch APIra zuzeneko sarbidea

Stack Management

Index Pattern / Data View sortu

ILM (Index Lifecycle Mgmt)

Erabiltzaileak eta Rolak

Snapshot Politikak

Alerting eta ML

Threshold-oinarritutako alertak

Anomalia detekzioa (ML)

Email/Slack jakinarazpenak

Watcher (Enterprise)

Kibana: :5601 | Elasticsearch APIrako kotsola integratua

Discover eta KQL

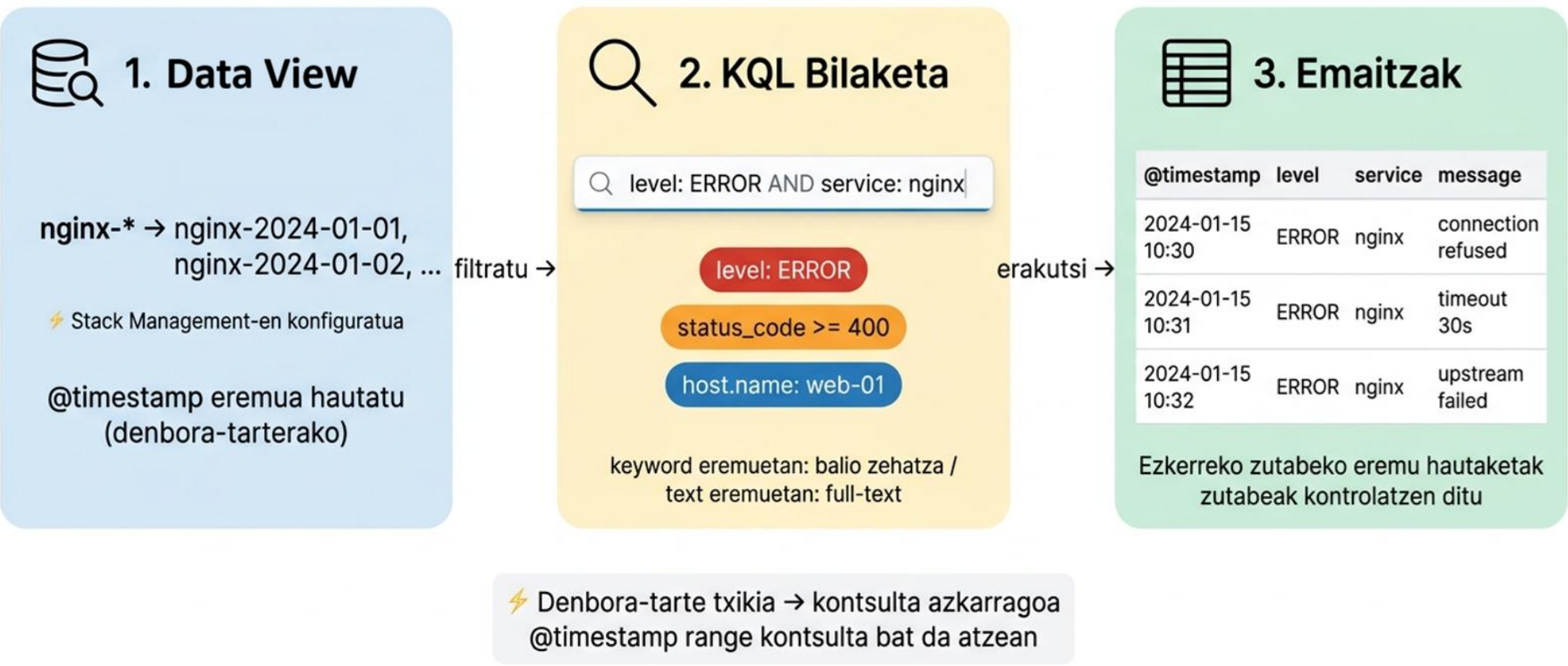
■ Discover

Elasticsearch-eko dokumentuak esploratu eta iragazteko Kibana-ko tresna.

■ Hiru urrats:

Data View → KQL → Emaizak

Kibana Discover: Nola Funtzionatzen Du



KQL · Kibana Query Language

```
●●● kql.txt python

1  # Balio zehatza
2  kategoria: "Informatika"
3
4  # Zenbakizko tartea
5  prezioa >= 100
6
7  # Baldintzak konbinatu
8  kategoria: "Informatika" AND prezioa < 500
9
10 # Baldintza baztertu
11 NOT kategoria: "Papergintza"
```

AND / OR / NOT

AND → bi baldintzak bete

OR → gutxienez bat bete

NOT → baldintza baztertu

Query DSL → kontsulta osoak eta konplexuagoak

KQL → Kibana-n iragazketa azkarra eta eroso

Discover eta KQL – Datuak esploratzen

HELBURUA: **produktuak** indizeko datuak **Discover-en** esploratu eta **KQL** bidez iragaztea.

PAUSOAK:

1. Ireki Discover.

- Joan **Kibana** → **Discover** atalera.
- Aukeratu **produktuak** indizeari dagokion **Data View-a**.
- Begiratu eskuragarri dauden dokumentuak eta eremuak.

PISTA: **produktuak** Data View-a ez badago, sortu:

- **Stack Management** → **Data Views** → **Create data view**
- Index pattern: **produktuak**
- Produktuek ez dute **@timestamp** eremurik.

2. Prestatu emaitzen taula.

- Gehitu zutabe hauek: **izena**, **kategoria**, **prezioa**

PISTA: Discover-en ezkerreko eremu-zerrendan bilatu eremua eta gehitu taulara.

3. Egin KQL bilaketa hauek:

- Informatika kategoriako produktuak.
- 100€ baino garestiagoak.
- Informatika kategoriakoak **ETA** 500€ baino merkeagoak
- Papergintza kategoriakoak **EZ** diren produktuak.

KQL GOGORATU:

- Balio bat bilatzeko → **eremua**: "**balioa**"
- Konparazioetarako → **>**, **<**, **>=**, **<=**
- Baldintzak konbinatzeko → **AND**, **OR**, **NOT**

4. Zer antzekotasun dago KQL bilaketa hauen eta Query DSL bidez egindako kontsulten artean?

Lens: datuetatik grafikoetara

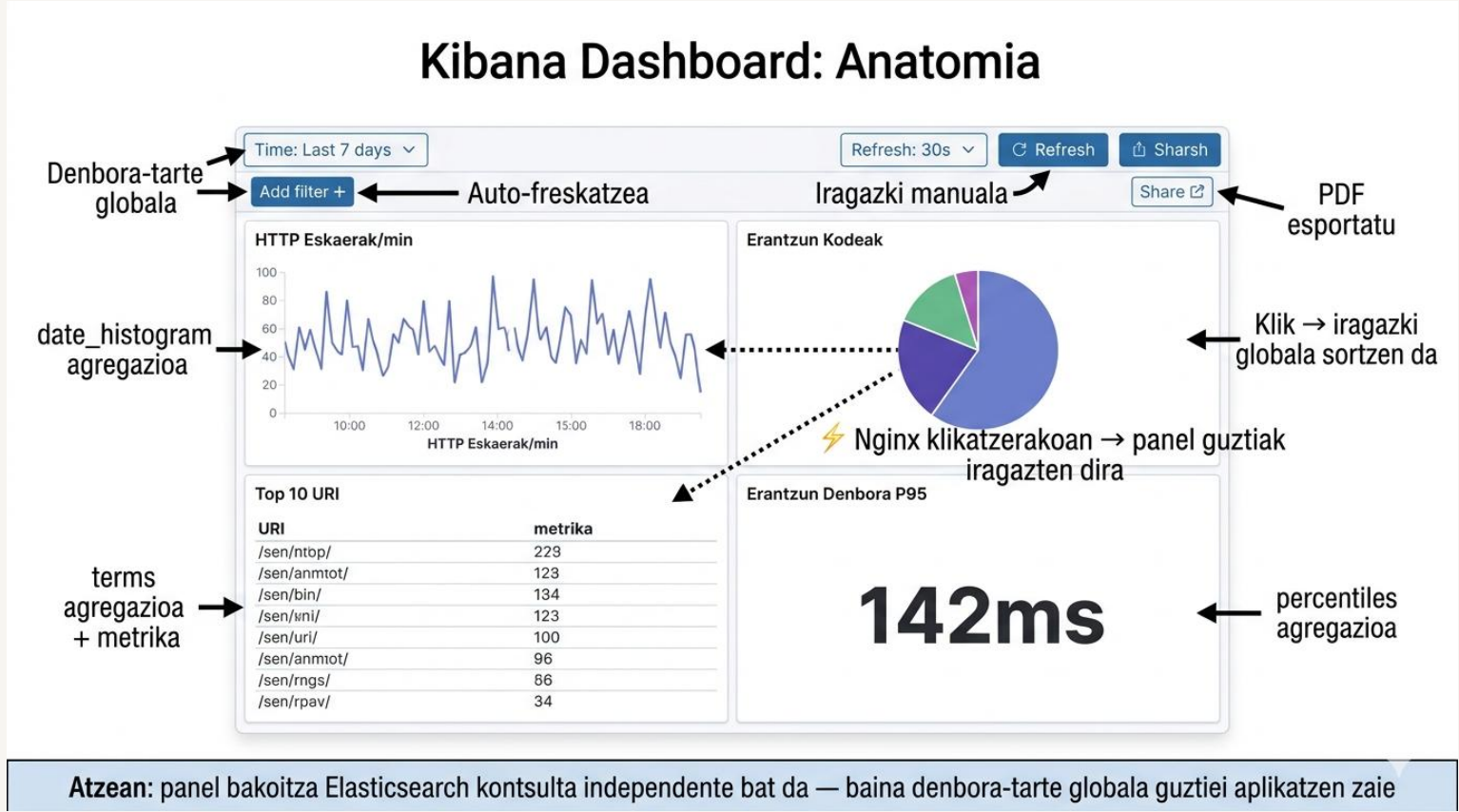
Kibana Visualization → Elasticsearch-eko datuetatik **bistaratzeak** sortzeko tresna grafikoa.

- **Drag & drop** → eremuak arrastatu, koderik gabe
- **Metrikak** → Count, Average, Sum, Min, Max...
- **Taldekatzea** → kategoriaren, zerbitzuaren... arabera
- **Grafiko motak** → barrak, lerroak, donut, metrikak...

IDEIA GAKOA: Lens-en grafiko baten atzean **Elasticsearch** agregazioak daude.

— DASHBOARD: BISTARATZEA

Dashboard: bistaratzeak panel batean



Dashboard batek bistaratzeak konbinatzen ditu:

- **Panelak konbinatu** → ikuspegi orokorra lortzeko.
- **Denbora-tarte globala** → panel guztiei batera aplikatzen zaie.
- **Iragazketa interaktiboa** → panel batean klik → panel guztiak iragazi.
- **Auto-freskatzea** → datuak automatikoki eguneratu monitorizaziorako.

ATZEAN: panel bakoitzak Elasticsearch kontsultatzen du; iragazki globalak kontsulta guztiei aplikatzen zaizkie.

Lens + Dashboard – Produktuen dashboard-a

HELBURUA: Lens (Visualization) erabiliz bistaratzeak sortu eta **Dashboard** batean konbinatzea.

PAUSOAK:

1. Sortu 1. bistaratzea → Produktuak kategoriaren arabera

- Joan **Visualize Library** → **Create visualization** → **Visualization**
- Erabili **kategoria** eremua
- Erakutsi **produktu kopurua** kategoria bakoitzean
- Erabili **barra-diagrama**
- Gorde: **Produktuak kategoriaren arabera**

PISTA: produktu kopurua kalkulatzeko erabili **Count of records**.

kategoria eremuak produktuak kategoriaka taldekatzeko balio du.

2. Sortu 2. bistaratzea → Batez besteko prezioa kategoriaren arabera

- Erabili **kategoria** → taldekatzea
- Erabili **prezioa** → **Average**
- Erabili **barra-diagrama**
- Gorde: **Batez besteko prezioa kategoriaren arabera**

PISTA: oraingoan ez dugu produktuak zenbatu nahi:

kategoria bakoitzeko **prezioa** eremuaren **batezbestekoa** kalkulatu nahi dugu.

3. Sortu Dashboard-a

- Sortu Dashboard berria (**Dashboards** → **Create a dashboard**)
- Gehitu gordetako **bi bistaratzeak**
- Aldatu panelen **tamaina eta kokapena**
- Gorde: **Produktuen Dashboard-a**

4. Probatu interaktibitatea

- Egin klik grafiko bateko **kategoria batean**
- Begiratu zer gertatzen den beste panelarekin
- Identifikatu sortutako **iragazkia**
- Kendu iragazkia eta egiaztatu emaitza

Lens + Dashboard – Salmenten dashboard-a

HELBURUA: Emandako salmenta-dataset bat (*salmentak_kibana.csv*) Elasticsearch-era inportatu, datuak aztertu, hainbat bistaratze sortu eta **Dashboard interaktibo batean konbinatzea**.

PAUSOAK:

1. Inportatu dataset-a.

Inportatu *salmentak_kibana.csv* Elasticsearch-era (Machine Learning → Data Visualizer → Upload data files).

- Indizearen izena: **salmentak**
- Egiaztatu dokumentuak behar bezala kargatu direla.
- Sortu/egiaztatu **salmentak** indizeari dagokion **Data View**.
- Ziurtatu **data** eremua **Date** motakoa dela.

PISTA: fitxategia kargatzerakoan begiratu Kibana-k eremu bakoitzari esleitutako datu mota.

2. Sortu bistaratzeak

Sortu eta gorde ondorengo bistaratzeak. **Zuek erabaki behar duzue zer eremu eta agregazio erabili behar diren.**

a. Salmentak kategoriaren arabera

Erakutsi kategoria bakoitzean **saldutako unitate guztien kopurua**. Erabili barra-diagrama.

b. Diru-sarrerak hirika

Erakutsi hiri bakoitzak sortutako **diru-sarrera guztiak**. Erabili barra-diagrama.

c. Salmenta-kanalen banaketa

Erakutsi salmenten banaketa **Denda / Web** kanalen artean. Erabili donut edo pie-diagrama

d. Diru-sarreraren bilakaera denboran

Erakutsi **diru-sarrera guztien bilakaera denboran**. Erabili lerro-diagrama eta **data** eremua.

e. Produktuen batez besteko prezioa

Erakutsi produktu bakoitzaren **batez besteko prezioa**. Erabili barra-diagrama.

PISTA: pentsatu kasu bakoitzean zer behar duzuen: **Count, Sum edo Average?**

Eta zein eremuk egiten duen **taldekatzea**.

Lens + Dashboard – Salmenten dashboard-a

HELBURUA: Emandako salmenta-dataset bat (*salmentak_kibana.csv*) Elasticsearch-era inportatu, datuak aztertu, hainbat bistaratze sortu eta **Dashboard interaktibo batean konbinatzea**.

PAUSOAK:

3. Sortu Dashboard-a

Sortu Dashboard berria: **Salmenten Dashboard-a**

Gehitu aurreko **5 bistaratzeak** eta antolatu informazioa modu argian.

Dashboard-ak gutxienez izan behar ditu:

- **5 bistaratzeak**
- **Izenburu argiak**
- **Panelen tamaina eta kokapen egokia**
- Informazioa erraz interpretatzeko antolaketa

4. Probatu Dashboard-aren interaktibitatea

Dashboard-a prest dagoenean: Egin klik hiri batean; begiratu gainerako grafikoak nola aldatzen diren; kendu iragazkia; iragazi **kategoria** edo **kanal** baten arabera eta aztertu emaitza.

5. Azken analisia

1. Zein kategoriatan saldu dira unitate gehien?
2. Zein hirik sortu du diru-sarrera handiena?
3. Zein kanaletan egin dira salmenta gehiago: Web edo Denda?
4. Zein produktuk du batez besteko preziorik handiena?
5. Aukeratu hiri bat eta aplikatu iragazkia. Zer ondorio interesgarri atera dezakezu hiri horretako salmentei buruz?

Dev Tools: Elasticsearch-ekin zuzenean

```
●●● devtools.http python

1  # Bilaketa kontsulta
2  GET /produktuak/_search
3  {
4  "query": { "match": { "izena": "koaderno" } }
5  }
6
7  # Kluster egoera
8  GET /_cluster/health
9
10 # Indizeak ikusi
11 GET /_cat/nodes?v
12
13 # Indize bat sortu / ezabatu
13 PUT /test-indizea
14 DELETE /test-indizea
```

EXEKUTATU

CTRL+ENTER → Kontsulta exekutatzeko lasterbidea.

Emitza → eskuineko panelean JSON formatuan agertzen da

LAGUNTZA

Autocompletatzea → API, indize eta eremuekin laguntza.

Sintaxi-akatsak identifikatzen laguntzen du.

_CAT API

Klusterraren informazioa **formatu irakurgarrian** ikusteko.

/_cat/indices?v → indizeak, dokumentu kopurua, tamaina, egoera...

Discover/Lens → interfaze grafikoa

Dev Tools → **Elasticsearch API-a zuzenean**

ILM — datuen bizi-zikloa

Indizeen **bizi-zikloa** automatikoki
kudeatzen du.

Arazoa: logak etengabe metatzen dira
→ biltegitratze-beharra eta kostua
handitu.

Soluzioa: datuak erabileraren arabera
fasez aldatu:

Hot → Warm → Cold → Delete

ILM — Index Lifecycle Management



Datu berriak → azkarragoak eta garestiagoak

Datu zaharrak → merkeagoak eta gutxiago erabiltzen dira

ILM Faseak · Ezaugarri eta Helburuak

FASEA	DENBORA *	EZAUGARRIAK	HELBURUA
Hot	0–7 egun	Idazketa + irakurketa aktiboa, errendimendu handia	Azken logak, arazketa
Warm	7–30 egun	Irakurketa nagusiki, erabilera txikiagoa	Datu historikoak
Cold	30–90 egun	Gutxitan kontsultatu, baliabide gutxiago	Artxibatzea, auditoretza
Delete	90 egun	Automatikoki ezabatu	Lekua eta kostua murriztu

* Denborak adibideko ILM politikarenak dira; konfiguragarriak dira.

ILM politika Kibana-n

HELBURUA: Kibana erabiliz, enpresa baten logen bizi-zikloa automatikoki kudeatuko duen ILM politika sortzea eta web-logs-* indize berriei aplikatzea.

Enpresa batek egunero bere **web zerbitzarien logak** jasotzen ditu Elasticsearch-en:

- Azken **7 egunetako** logak maiz erabiltzen dira.
- **7-30 egunekoak** noizean behin kontsultatzen dira.
- **30-90 egunekoak** auditoretzarako gordetzen dira.
- **90 egunetik gorakoak** ez dira behar.

- Index pattern: **web-logs-***
- Create data stream: **desaktibatuta**
- Lotu **web-logs-policy** ILM politika template-ari (*Index settings* atalean):

```
{  
  "index.lifecycle.name": "web-logs-policy"  
}
```

1. Sortu **web-logs-policy** izeneko **ILM politika** Kibana-n (*Stack Management → Index Lifecycle Policies*)

- HOT: hasieratik
- WARM: 7 egunetik aurrera
- COLD: 30 egunetik aurrera
- DELETE: 90 egunetik Aurrera

2. Sortu **web-logs-template** izeneko **Index Template** bat (*Stack Management → Index Management → Index Templates*):

3. Egiaztatu politika eta template-a behar bezala sortu direla.

4. Bihar **web-logs-2026.10.02** indizea sortzen bada, ILM politika eskuz esleitu beharko diogu? Zergatik?

5. Zergatik ez ditugu log guztiak Hot fasean mantentzen?

6. Zer arazo izango genuke Delete faserik ez balego?

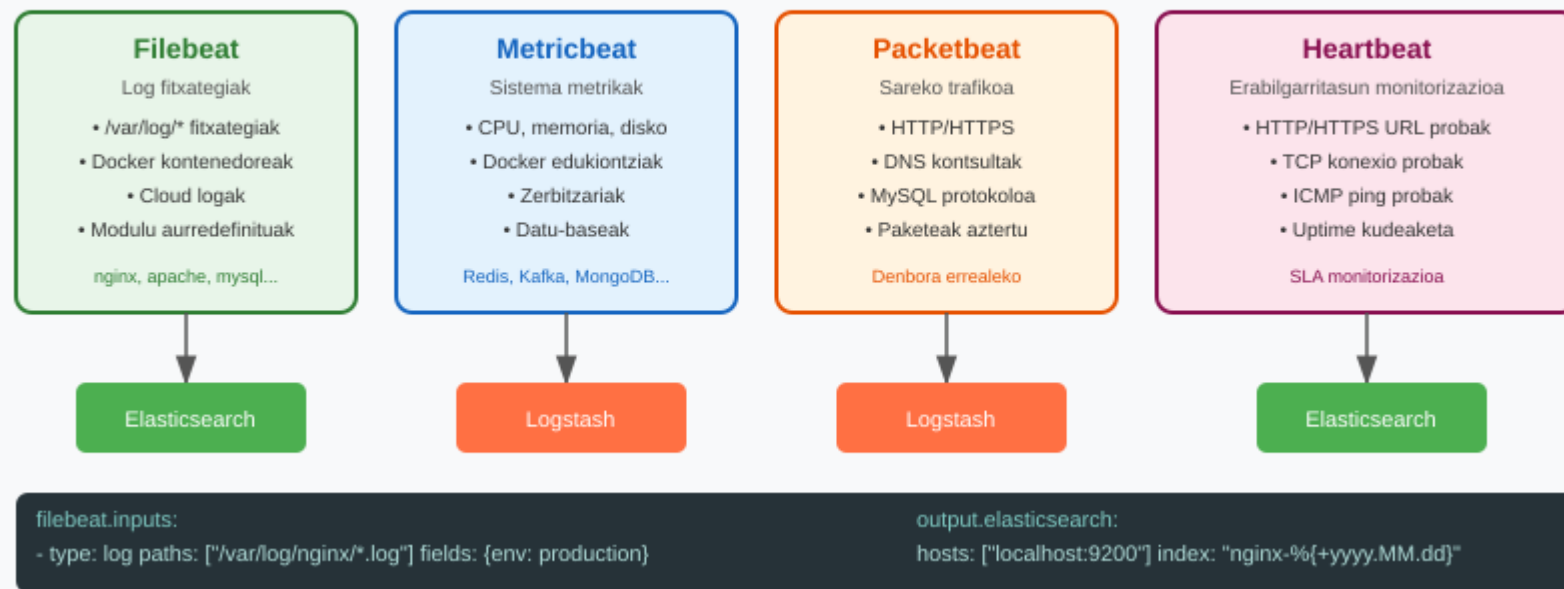
05

Ingesta: Beats eta Logstash

— AGENTE ARINAK

Beats

Beats: Datu Bilketa Agenteak



Beats datuak jatorrian bildu eta bidaltzen dituzten agente arinak dira.

- **Jatorrian instalatzen dira** – zerbitzariaren lana ahalik eta gutxien oztopatzeko
- **Filebeat** → logak, **Metricbeat** → CPU/RAM/diskoa, **Packetbeat** → sareko trafikoa
- **Heartbeat** → erabilgarritasuna, **Winlogbeat** → Windows event-ak, **Auditbeat** → auditoretza
- Datuak **Elasticsearch-era zuzenean** edo **Logstash-en bidez** bidal ditzakete

Aukeratu Beat egokia

Adierazi zein Beat erabiliko zenukeen:

1. Linux zerbitzari baten **/var/log/nginx/access.log** jasotzeko.
2. Zerbitzari baten CPU eta RAM erabilera monitorizatzeko.
3. Windows-eko Event Viewer-eko gertaerak jasotzeko.
4. Webgunea erabilgarri dagoen egiaztatzeko.
5. Sareko trafikoa aztertzeko.
6. Linux zerbitzari batean segurtasun-auditoretza egiteko.

Diseinatu ingesta

EGOERA: Enpresa batek honako azpiegitura dauka:

- 20 Linux web zerbitzari
- Zerbitzari bakoitzak NGINX logak sortzen ditu
- Zerbitzarien CPU eta RAM ere monitorizatu nahi dira
- Webgunea kanpotik erabilgarri dagoen jakin nahi dute
- Datu guztiak Elasticsearch-en zentralizatu nahi dituzte.

Erantzun:

1. Zein Beat instalatuko zenuke web zerbitzarietan logak jasotzeko?
2. Zein Beat erabiliko zenuke CPU/RAM jasotzeko?
3. Zein Beat erabiliko zenuke webgunea erabilgarri dagoen egiaztatzeko?
4. Nora bidal ditzakete Beats-ek datuak?

Filebeat konfigurazioa interpretatu

EGOERA: Enpresa bateko Filebeat-ek konfigurazio hau dauka (*filebeat.yml*):

```
filebeat.inputs:
  - type: filestream
    id: nginx-logs
    paths:
      - /var/log/nginx/*.log

output.elasticsearch:
  hosts: ["http://elasticsearch:9200"]
```

Erantzun:

1. Zer datu mota ari da jasotzen Filebeat?
2. Zein karpetatik?
3. ***.log** zer esan nahi du?
4. Nora bidaltzen ditu datuak?
5. Logstash erabiltzen ari gara konfigurazio honetan?

Filebeat: Output-a aldatu

EGOERA: Aurreko ariketan Filebeat-ek NGINX logak irakurri eta zuzenean Elasticsearch-era bidaltzen zituen:

NGINX → Filebeat → Elasticsearch

Orain enpresak datuak lehenengo Logstash-era bidali nahi ditu:

NGINX → Filebeat → Logstash → Elasticsearch

Aurreko ariketako *filebeat.yml* konfigurazioa abiapuntu hartuta:

1. Aldatu output-a, datuak Elasticsearch-era bidali beharrean Logstash-era bidaltzeko.

Erabili:

- **Host-a: logstash**
- **Portua: 5044**

Idatzi konfigurazio berria. PISTA:

```
output.logstash:  
  hosts: [...]
```

2. Erantzun:

- Zein konfigurazio-zati aldatu duzu?
- Filebeat-ek Elasticsearch-era zuzenean bidaltzen al ditu datuak orain?
- Zein da Filebeat-en hurrengo osagaia pipeline-an?

Beats kasu praktikoa: Apache logak Filebeat-ekin

HELBURUA: Apache web zerbitzari batek sortutako logak **Filebeat** erabiliz bildu, **Elasticsearch-era** bidali eta **Kibana** bidez aztertzea.

Eraiki beharreko datu-fluxua:

Apache → Filebeat → Elasticsearch → Kibana

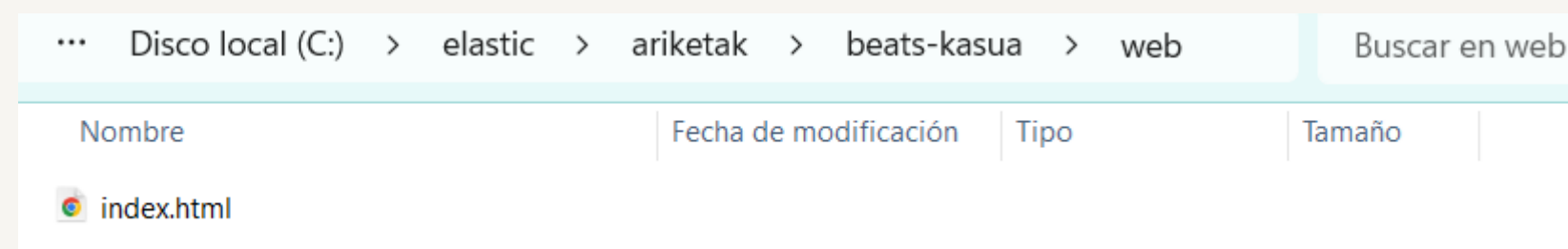
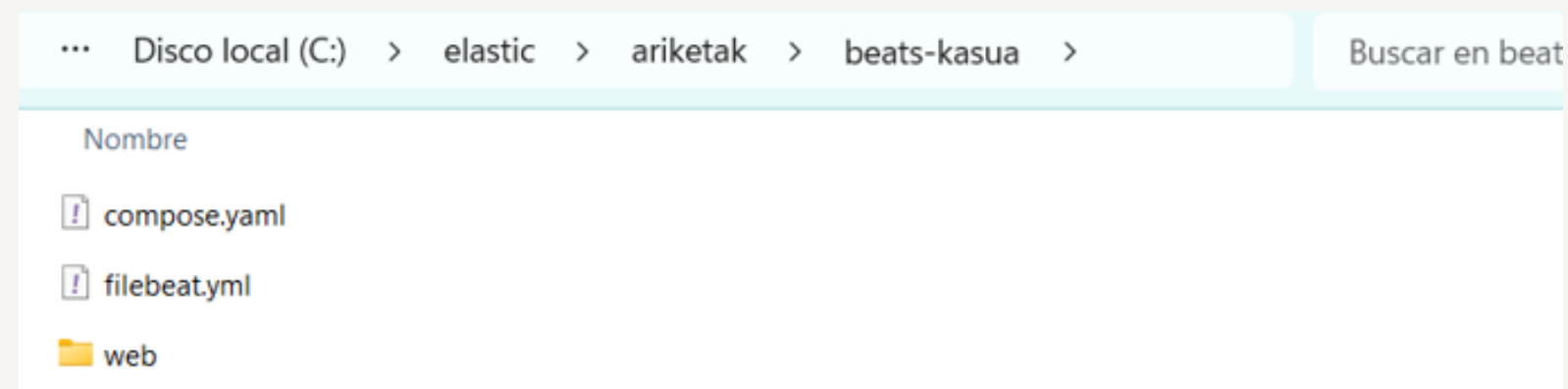
Ariketaren amaieran gai izan behar duzu azaltzeko:

- Nork sortzen ditu logak?
- Nork jasotzen eta bidaltzen ditu?
- Non gordetzen dira?
- Nola aztertzen dira?

Beats kasu praktikoa: Apache logak Filebeat-ekin

Lan ingurunea prestatu:

Sortu **C:\elastic\ariketak\beats-kasua** karpeta.



Beats kasu praktikoa: Apache logak Filebeat-ekin

Sortu **index.html** fitxategia web karpeta barruan. Apache-k orri hori erakutsiko du.

Adibidez:

```
web > </> index.html > html
1  <!DOCTYPE html>
2  <html lang="eu">
3
4  <head>
5      <meta charset="UTF-8">
6      <title>Beats praktika</title>
7  </head>
8
9  <body>
10
11      <h1>Apache + Filebeat</h1>
12
13      <p>Web-orri hau Beats praktikaren barruan sortu dugu.</p>
14
15  </body>
16
17  </html>
```

Beats kasu praktikoa: Apache logak Filebeat-ekin

Sortu **filebeat.yml** fitxategia.

Filebeat-ek:

- Apache-ren **access_log** irakurri behar du.
- Datuak zuzenean Elasticsearch-era bidali behar ditu.

Adibidez:

```
Y filebeat.yml
1  filebeat.inputs:
2
3    - type: filestream
4      id: apache-access
5      enabled: true
6      paths:
7        - /var/log/apache/access_log
8
9  output.elasticsearch:
10   hosts: ["http://elasticsearch:9200"]
```

— 13. PRAKTIKA

Beats kasu praktikoa: Apache logak Filebeat-ekin

Sortu **compose.yaml** fitxategia eta kopiatu konfigurazio hau:

Begiratu konfigurazioari eta erantzun:

1. Zein lau zerbitzu sortzen dira?
2. Zein portutan dago eskuragarri?
 - Elasticsearch?
 - Kibana?
 - Apache?
3. Zertarako erabiltzen da apache-logs volume-a?
4. Zergatik muntatzen da volumen bera path desberdinetan Apache-n eta Filebeat-en?
5. Zer esan nahi du Filebeat-en: **:ro**?
6. Zer egiten du Apache-ren **command** blokeak?

```
compose.yaml
1  services:
2
3      elasticsearch:
4          image: docker.elastic.co/elasticsearch/elasticsearch:9.1.4
5          container_name: beats-elasticsearch
6          environment:
7              - discovery.type=single-node
8              - xpack.security.enabled=false
9          ports:
10             - "9200:9200"
11
12      kibana:
13          image: docker.elastic.co/kibana/kibana:9.1.4
14          container_name: beats-kibana
15          environment:
16              - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
17          ports:
18              - "5601:5601"
19          depends_on:
20              - elasticsearch
21
22      apache:
23          image: httpd:2.4
24          container_name: beats-apache
25          ports:
26              - "8080:80"
27          volumes:
28              - ./web:/usr/local/apache2/htdocs
29              - apache-logs:/usr/local/apache2/logs
30          command:
31              - /bin/sh
32              - -c
33              - |
34                  echo 'CustomLog "/usr/local/apache2/logs/access_log" common' >> /usr/local/apache2/conf/httpd.conf
35                  exec httpd-foreground
36
37      filebeat:
38          image: docker.elastic.co/beats/filebeat:9.1.4
39          container_name: beats-filebeat
40          user: root
41          command: filebeat -e --strict.perms=false
42          volumes:
43              - ./filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
44              - apache-logs:/var/log/apache:ro
45          depends_on:
46              - elasticsearch
47              - apache
48
49  volumes:
50      apache-logs:
```

Beats kasu praktikoa: Apache logak Filebeat-ekin

Ingurunea abiarazi (Docker). Martxan egon behar dira:

- **beats-apache**
- **beats-filebeat**
- **beats-elasticsearch**
- **beats-kibana**

Probatu zerbitzuak:

- Elasticsearch: **http://localhost:9200**
- Apache: **http://localhost:8080**

Sortu HTTP eskaerak:

Egin gutxienez 10 eskaera:

- Baliabide zuzena:
 - **http://localhost:8080**
- Existitzen ez diren baliabideak:
 - **http://localhost:8080/proba**
 - **http://localhost:8080/ikasleak**
 - **http://localhost:8080/ez-dago**

Logetan egoera-kode hauek ager daitezke:

- **200 – OK**
- **304 – Not Modified**
- **404 – Not Found**

Beats kasu praktikoa: Apache logak Filebeat-ekin

Apache-ren logak ikusi:

```
docker exec beats-apache cat /usr/local/apache2/logs/access_log
```

Adibidez: 172.x.x.x - - [...] "GET /proba HTTP1.1" 404...

Identifikatu: *bezeroaren IP-a, HTTP metodoa, eskatutako baliabidea, HTTP bertsioa eta status code-a*

Filebeat-ek log bera ikus dezake?

```
docker exec beats-filebeat cat /var/log/apache/access_log
```

Filebeat egiaztatu: `docker logs --tail 50 beats-filebeat`

Beharrezkoa bada, probate Elasticsearch output-a: `docker exec beats-filebeat filebeat test output -e --strict.perms=false`

Espero dena: *talk to server... OK*

Beats kasu praktikoa: Apache logak Filebeat-ekin

Elasticsearch-en egiaztatu:

Ikusi indizeak: http://localhost:9200/_cat/indices?v

Beharrezkoa bada, bilatu Filebeat-en datuak: http://localhost:9200/_resolve/index/filebeat*?pretty

Kontsultatu dokumentuak: http://localhost:9200/filebeat-*/_search?pretty

Galdera: Apache-ren jatorrizko log-lerroa zein eremutan agertzen da?

Beats kasu praktikoa: Apache logak Filebeat-ekin

Kibana:

Ireki: <http://localhost:5601>

Joan: **Discover**

Beharrezkoa bada, sortu Data View: **filebeat-***, Time field: **@timestamp**

Zabaldu dokumentu bat eta bilatu: **message**

Aurkitzen al dituzu:

- 200 edo 304 erantzunak?
- 404 erantzunak?

Beats kasu praktikoa: Apache logak Filebeat-ekin

Azken galderak:

1. Zein da pipeline honetako datuen jatorria?
2. Zer funtzio betetzen du Filebeat-ek?
3. Non gordetzen dira logak Filebeat-ek bidali ondoren?
4. Zer funtzio betetzen du Kibana-k?
5. Zertarako erabili dugu Docker volume-a?
6. Zer desberdintasun dago HTTP eskaera eta erantzun baten artean?
7. Zer esan nahi dute 200, 304 eta 404 egoera-kodeek?
8. Zergatik erabiltzen dugu Filebeat-en: **elasticsearch:9200** eta ez **localhost:9200**?
9. Osatu datu-fluxua: _____ → _____ → _____ → _____

Logstash: testu gordina → datu egituratuak

ARAZOA:

Log-lerro bat testu gordina da:

```
192.168.1.105 "GET /api" 500 1234
```

Informazioa badago, baina **ez dago eremuetan banatuta**.

Ezin dugu zuzenean hau egin:

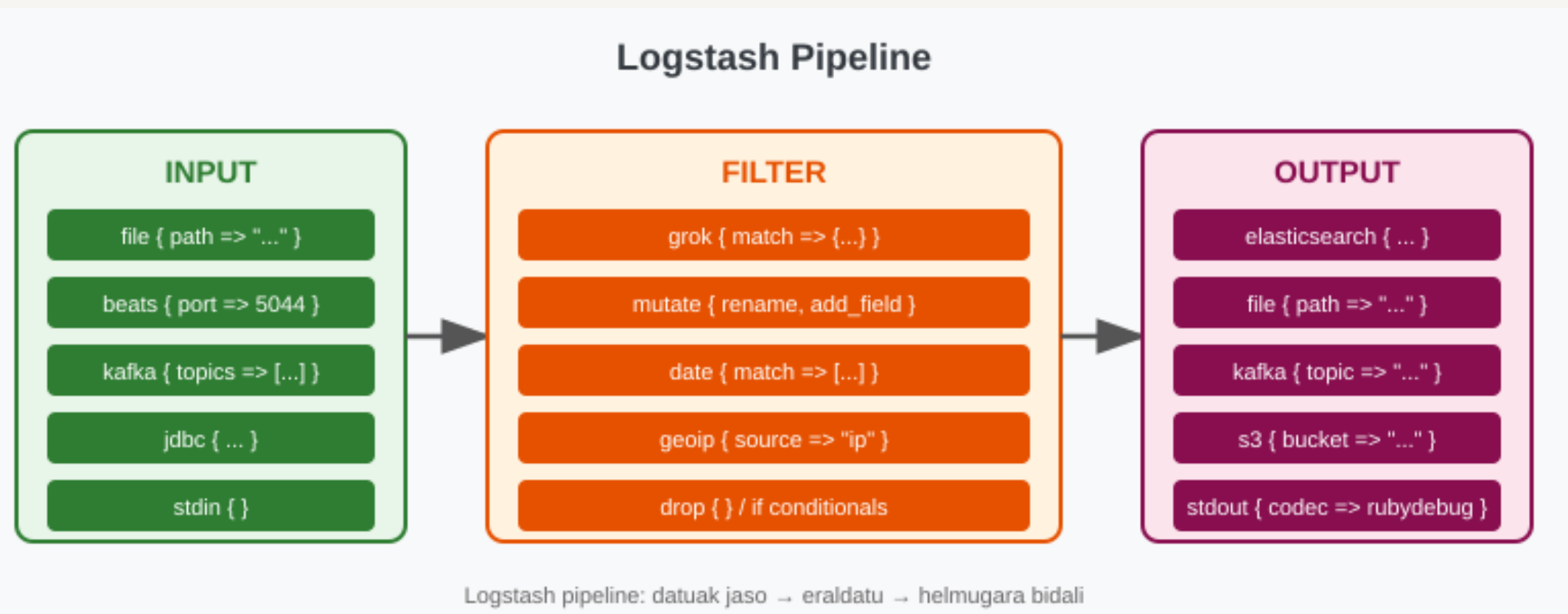
status_code: 500

LOGSTASH:

Logstash-ek testu gordina **parseatu eta eremu egituratuetan banatzen du**:

```
bezero_ip:      192.168.1.105
metodoa:        GET
request:        /api
status_code:    500
bytes:          1234
```

Testu gordina → eremu egituratuak → bilaketa eta analisisa



— PIPELINE

Input → Filter → Output

Logstash pipeline batek hiru fase nagusi ditu:

- **INPUT – datuak jaso:**
Datuak nondik datozen definitzen du.
Beats, fitxategiak, Kafka, JDBC...
- **FILTER – datuak eraldatu:**
Jasotako datuak parseatu, garbitu edo aberasten ditu.
Grok, Date, GeoIP, Mutate...
- **OUTPUT – datuak bidali:**
Prozesatutako datuen helmuga definitzen du.
Elasticsearch, Kafka, fitxategiak, S3...

Pipeline-a .conf fitxategi batean definitzen da:

```
input {  
  ...  
}  
  
filter {  
  ...  
}  
  
output {  
  ...  
}
```

Lehen Logstash pipeline-a

HELBURUA: Logstash pipeline sinple bat sortu eta **input** → **output** fluxua praktikan ikustea.

Sortu pipeline bat, **teklatutik idatzitako testua jaso eta terminalean erakusteko:**

Teklatua → Logstash → Terminala

1. Sortu **lehen-pipeline.conf** fitxategia:

```
input {
  stdin { }
}

output {
  stdout {
    codec =>
    rubydebug
  }
}
```

2. Abiarazi Logstash konfigurazio hori erabiliz:

```
docker run --rm -it `
-v "${PWD}\lehen-pipeline.conf:/usr/share/logstash/pipeline/lehen-pipeline.conf:ro" `
docker.elastic.co/logstash/logstash:9.1.4 `
logstash -f /usr/share/logstash/pipeline/lehen-pipeline.conf
```

3. Pipeline-a martxan dagoenean, idatzi: **Kaixo Logstash**

4. Begiratu Logstash-ek sortutako event-a eta identifikatu:

- **message**
- **@timestamp**

5. Erantzun:

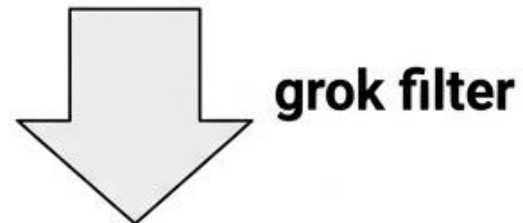
- Zein da pipeline honen **input-a**?
- Zein da **output-a**?
- Zergatik ez dago **filter** blokerik?
- Zer balio dauka **message** eremuak?
- Zer adierazten du **@timestamp** eremuak?

PISTA: **filter** blokea ez da derrigorrezkoa; datuak eraldatu behar direnean erabiltzen dugu.

grok: Testu Gordina → Eremu Egituratura

```
192.168.1.105 - - [15/Jan/2024:10:30:45 +0100] "GET /api/v1 HTTP/1.1" 200 1234
```

```
%{IP:bezero_ip} %{WORD:metodoa} %{URIPATH:request} HTTP/%{NUMBER:httpversion}  
%{NUMBER:status_code:int} %{NUMBER:bytes:int}
```



```
{  
  "bezero_ip": "192.168.1.105",  
  "metodoa": "GET",  
  "request": "/api/v1",  
  "httpversion": "1.1",  
  "status_code": 200,  
  "bytes": 1234  
}
```

`%{IP}` → IPv4/IPv6 helbidea

`%{WORD}` → Hitz bat

`%{URIPATH}` → URL path

`%{NUMBER}` → Zenbakia

`%{GREEDYDATA}` → Edozer (⚠️ astiro)



Kibana Grok Debugger:
Dev Tools → Grok Debugger.
Pattern garatu aurretik probatu!

— FILTER GAKOA

Grok — testu gordina → eremu egituratuak

Grok-ek testu bateko patroiak identifikatu eta balioak eremu izendatuetan gordetzen ditu.

Oinarrizko sintaxia:

```
%{PATROIA:eremu_izena}
```

Adibidez:

```
%{IP:bezero_ip}
```

IP helbidea identifikatu eta bezero_ip eremuan gorde

Oharra: irudiak Grok-en funtzionamendua modu sinplifikatuan erakusten du; log erreal batean pattern-ak lerroaren egitura osoa hartu behar du kontuan.

Grok Debugger

HELBURUA: Testu gordina Grok pattern baten bidez eremu egituratuetan banatzea.

1. Ireki Grok Debugger.

- Kibana-n: Dev Tools → Grok Debugger

2. Sartu log-lerro hau:

- Sample Data:

```
192.168.1.105 GET /api 500 1234
```

3. Sortu Grok pattern-a. Honako eremuak atera behar dituzu:

- **bezero_ip**
- **metodoa**
- **request**
- **status_code**
- **bytes**

PISTA: Erabil ditzakezun patroiak:

```
%{IP} · %{WORD} · %{URIPATH} · %{NUMBER}
```

4. Probatu.

Sakatu Simulate eta egiaztatu bost eremuak behar bezala sortzen direla.

Ondoren, probatu pattern bera beste log honekin:

```
10.0.0.25 POST /login 200 856
```

5. Erantzun:

1. Zein Grok pattern sortu duzu?
2. Pattern bera erabil daiteke bi logetarako?
Zergatik?
3. Zer gertatzen da logaren egitura aldatzen bada?

Grok Debugger

HELBURUA: Testu gordina Grok pattern baten bidez eremu egituratuetan banatzea.

6. Sortu beste Grok pattern bat.

- Sample Data:

```
10.0.0.25 POST /login 401 856 erabiltzailea1
```

7. Honako eremuak atera behar dituzu:

- **bezero_ip**
- **metodoa**
- **request**
- **status_code**
- **bytes**
- **erabiltzailea**

8. Sortu beste Grok pattern bat:

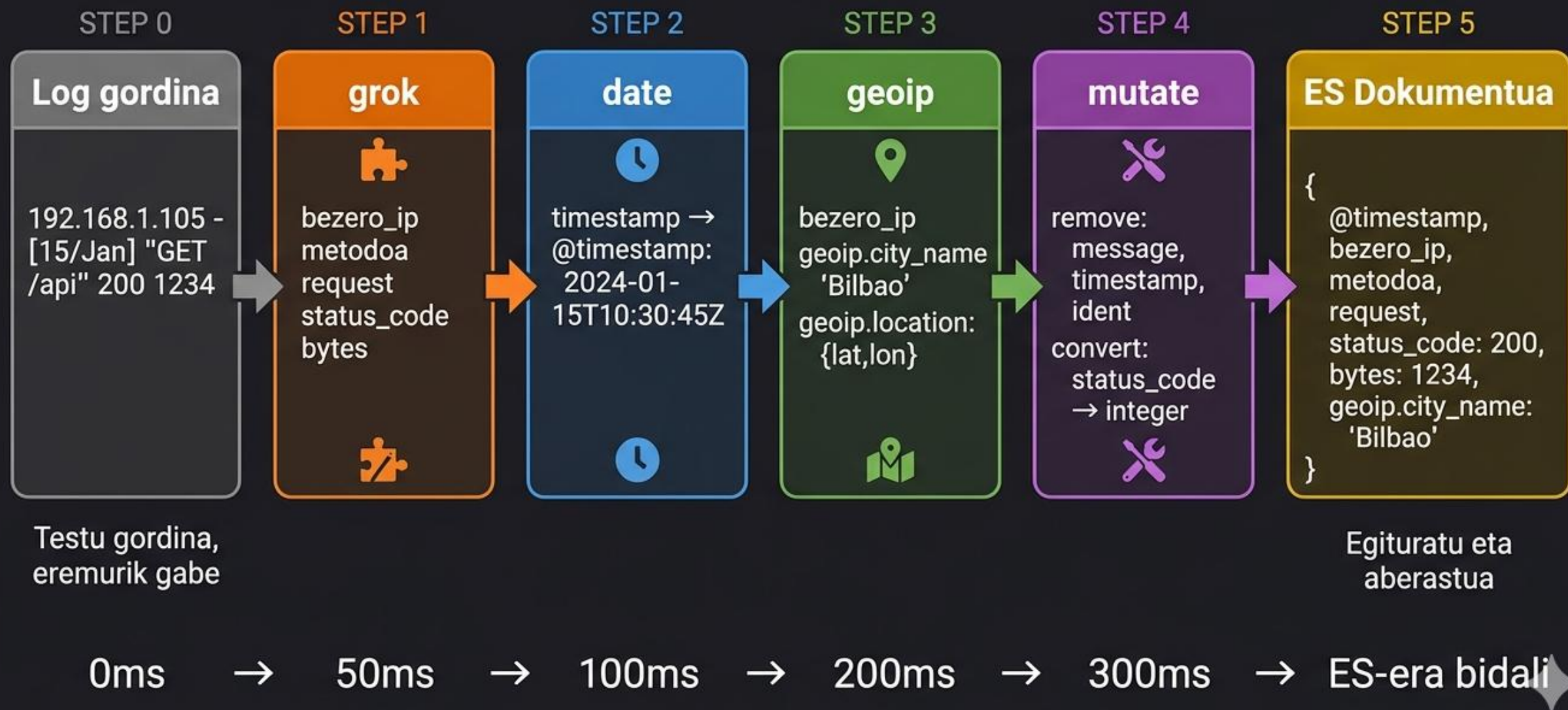
- Sample Data:

```
2026-10-04 10.0.0.25 POST /login 401 856
```

9. Honako eremuak atera behar dituzu:

- **data**
- **bezero_ip**
- **metodoa**
- **request**
- **status_code**
- **bytes**

Nginx Log → Filter Katea → ES Dokumentua



—FILTER KATEA

grok → date → geoip
→ mutate

Filter-ak bata bestearen atzetik aplikatzen dira; bakoitzak event-a eraldatu edo aberasten du.

- **Grok** → testua eremuetan banatu
- **Date** → logaren data **@timestamp** bihurtu
- **Geoip** → IP-ari kokapen geografikoa gehitu
- **Mutate** → eremuak garbitu, aldatu edo mota bihurtu

Emitza: Elasticsearch-era dokumentu **egituratu** eta **aberrastua** iristen da.

Logstash-en ohiko akatsa

date filter-a ahaztea

Log-ak bere data eta ordua darama, baina Logstash-ek ez du automatikoki jakiten testuko data hori event-aren benetako timestamp-a dela.

- **date filter GABE:** `@timestamp` = Logstash-ek event-a **prozesatu duen unea**
- **date filter-arekin:** `@timestamp` = logean adierazitako **gertaeraren unea**

GARRANTZITSUA

Logak bere timestamp-a badarama, date filter-ak hura parseatu eta `@timestamp`-en ezartzen du. Horrela, Kibana-ko denbora grafikoek gertaera benetan noiz gertatu zen erakusten dute.

Logstash-en ohiko akatsa

ADIBIDEA:

Logean:

29/Sep/2026 23:45:10

Logstash-ek prozesatu:

30/Sep/2026 09:15:32

@timestamp	
Date gabe	30/Sep 09:15:32
Date-rekin	29/Sep 23:45:10

Grok + Date

HELBURUA: Log batetik, **timestamp**, **bezero_ip**, **metodoa**, **request** eta **status_code** eremuak ateratzea eta **timestamp** → **@timestamp** bihurtzea.

1. Demagun log hau daukagula:

```
30/Sep/2026:10:30:45 192.168.1.105 GET /api 500
```

2. Grok pattern-a atera.

3. Date filter-a konfiguratu, dataren formatua kontuan izanda.

4. Osatu kodea, hurrengo kode honetatik abiatuz:

```
filter {
  grok {
    match => {
      "message" => "..."
    }
  }

  date {
    match => [ "timestamp", "..." ]
  }
}
```

Mutate. Event-a garbitu

HELBURUA: mutate filter-a erabiliz event baten eremuak **mota bihurtu, berrizendatu eta ezabatu.**

Eraiki pipeline hau: **JSON event-a → Logstash (mutate) → Terminala**

1. Sortu **mutate.conf** fitxategia:

(oharra: **codec => json** jartzen dugu terminalean JSON event oso bat sartuko dugulako)

```
input {
  stdin {
    codec => json
  }
}

filter {

  mutate {
    convert => { ... }
    rename => { ... }
    remove_field => [ ... ]
  }

}

output {
  stdout {
    codec => rubydebug
  }
}
```

Abiapuntuko event honetatik abiatuz:

```
{
  "bezero_ip": "192.168.1.105",
  "metodoa": "GET",
  "request": "/api",
  "status_code": "500",
  "bytes": "1234",
  "message": "192.168.1.105 GET /api 500 1234"
}
```

Egin transformazio hauek:

1. **status_code** eremua **integer** bihurtu
2. **bytes** eremua **integer** bihurtu
3. **request** eremuaren izena **uri** bihurtu
4. **message** eremua **ezabatu**

PISTA:

```
mutate {
  convert => { ... }
  rename => { ... }
  remove_field => [ ... ]
}
```

Mutate. Event-a garbitu

HELBURUA: mutate filter-a erabiliz event baten eremuak **mota bihurtu, berrizendatu eta ezabatu.**

Eraiki pipeline hau: **JSON event-a → Logstash (mutate) → Terminala**

2. Abiarazi Logstash konfigurazio hori erabiliz:

```
docker run --rm -it -v  
"${PWD}\mutate.conf:/usr/share/logstash/pipeline/mutate.conf:ro"  
docker.elastic.co/logstash/logstash:9.1.4 logstash -f  
/usr/share/logstash/pipeline/mutate.conf
```

3. Sartu event-a terminalean:

```
{"bezero_ip":"192.168.1.105","metodoa":"GET","request":"/api","status  
_code":"500","bytes":"1234","message":"192.168.1.105 GET /api 500  
1234"}
```

4. Egiaztatu emaitza. Amaieran, event-ak antzeko egitura izan behar du:

```
{  
  "bezero_ip": "192.168.1.105",  
  "metodoa": "GET",  
  "uri": "/api",  
  "status_code": 500,  
  "bytes": 1234,  
}
```

Log gordina → filter-katea

HELBURUA: Log gordina aztertu eta behar diren **Logstash filter-ak identifikatu eta ordenatzea.**

Demagun log-lerro hau daukagula:

```
2026-10-05 10:30:45 ERROR nginx 'Connection refused' client=203.0.113.5
```

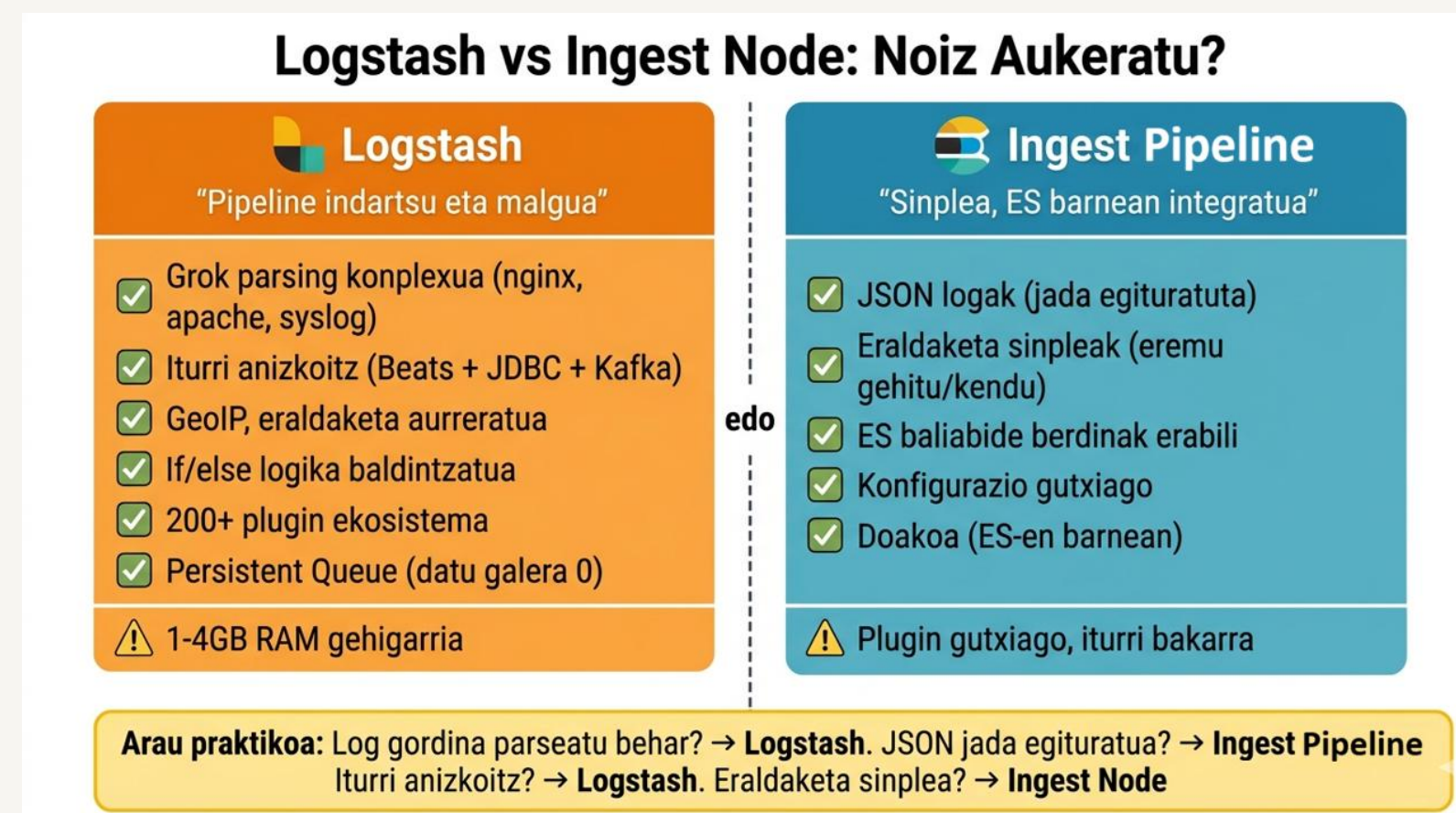
1. Zein eremu aterako zenituzke log honetatik?
2. Zein Logstash filter erabiliko zenituzke log hau prozesatzeko? Idatzi filter-ak ordena egokian.
3. Azaldu labur filter bakoitzak zer egingo lukeen log honekin.
4. Sortu Grok pattern bat, eremu hauek ateratzeko: *timestamp, maila, zerbitzua, mezua, bezero_ip*.
Probatu pattern-a **Grok Debugger-en**.
5. Idatzi **Date filter-aren konfigurazioa**, aurreko pausuan sortutako **timestamp** eremua parseatu eta event-aren **@timestamp** eremuan ezartzeko.
6. **GeoIP** erabiliko bazenu, zein eremuren gainean aplikatuko zenuke? Nola? Zer informazio gehituko luke?
7. **Mutate** filter-a beharrezkoa al da kasu honetan? Justifikatu. Zein egoeratan erabiliko zenuke?
8. Nolakoa izango litzateke Elasticsearch-era iritsiko den dokumentua?

Logstash vs Ingest Pipeline

Bi aukerek datuak eraldatu ditzakete, baina leku eta gaitasun desberdinekin.

- **Logstash** — Elasticsearch-etik kanpoko prozesamendua; iturri eta helmuga anitz, pipeline konplexuak eta plugin ugari.
- **Ingest Pipeline** — Elasticsearch-en barruan prozesatzen du; pipeline sinpleagoak eta Elasticsearch-era doazen datuak eraldatzeko egokia.
- **Logstash aukeratu** → Iturri/helmuga anitz, logika konplexua edo prozesamendu sendoagoa behar denean.
- **Ingest Pipeline aukeratu** → datuak Elasticsearch-era doazenean eta eraldaketa Elasticsearch-en bertan egin daitekeenean.

Ez dago aukera bakar “zuzena”: pipeline-aren beharren arabera da.



Aukeraketa: Logstash edo Ingest Pipeline?

Kasu bakoitzean aukeratu aukera egokiena eta justifikatu.

- A. Aplikazio batek JSON egituratua bidaltzen du Elasticsearch-era. Eremu berri bat gehitu behar da.
- B. Filebeat, Kafka eta datu-base batetik datuak jaso behar dira eta Elasticsearch-era bidali.
- C. Elasticsearch-era doazen dokumentuetan eremu baten izena aldatu behar da.
- D. Kafka-tik datuak jaso, baldintza desberdinak aplikatu eta bi helmugatara bidali behar dira.

Pipeline akastun bat konpondu

Demagun pipeline hau daukagula:

```
input {
  beats {
    port => 5044
  }
}

filter {
  grok {
    match => {
      "message" =>
        "%{IP:bezero_ip} %{WORD:metodoa} %{URIPATH:request}
        %{NUMBER:status_code}"
    }
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
  }
}
```

Jasotako loga:

```
05/Oct/2026:14:32:10 83.45.120.10 GET /api 500
```

Aurkitu arazoak:

1. Grok pattern-ak funtzionatuko du?
2. Zer zati falta zaio pattern-ari?
3. Zer filter falta da **@timestamp** zuzena izateko?
4. **status_code** zein motatakoa izatea komeni da?
5. Nola probatuko zenuke Grok pattern-a pipeline-a abiarazi aurretik?
6. Zuzendu pipeline-a.

Pipeline akastun bat konpondu. ERRONKA: Probatu

1. Sortu lan-karpeta.
2. Sortu web.log fitxategia. Idatzi hiru log hauek:
4. Sartu karpetan pipeline.conf fitxategia.
5. Logstash contaner-a abiarazi.
6. Egiaztatu emaitzak.

```
05/Oct/2026:14:32:10 83.45.120.10 GET /api 500
05/Oct/2026:14:33:20 192.168.1.15 POST /login 200
05/Oct/2026:14:34:45 10.0.0.8 GET /produktuak 404
```

3. Sortu compose.yaml fitxategia.

```
services:
  > Run Service
  logstash:
    image: docker.elastic.co/logstash/logstash:9.3.1
    container_name: iabd-logstash-20

    environment:
      XPACK_MONITORING_ENABLED: "false"

  volumes:
    - ./pipeline.conf:/usr/share/logstash/pipeline/pipeline.conf:ro
    - ./web.log:/usr/share/logstash/datuak/web.log:ro
```

Log berriak prozesatu

Aurreko praktikan hiru log prozesatu ditugu. Orain web zerbitzariak eskaera gehiago jaso ditu.

1. Gehitu log hauek web.log fitxategian:

```
06/Oct/2026:09:15:20 192.168.1.20 GET /hasiera 200
06/Oct/2026:09:16:35 10.0.0.15 POST /login 401
06/Oct/2026:09:17:50 172.16.0.8 DELETE /produktuak 403
```

2. Egin beharrekoa:

- Abiarazi berriro Logstash.
- Egiaztatu log berriak ondo prozesatzen direla.
- Begiratu status_code eremuaren balioak.
- Identifikatu errorea adierazten duten eskaerak.

Galdera: Pipeline-a aldatu behar izan duzu log berriak prozesatzeko?

HTTP erroreak identifikatu

EGOERA: Enpresak erroreak sortzen dituzten HTTP eskaerak identifikatu nahi ditu. Aurreko pipeline-a erabiliz:

1. Gehitu if baldintza bat filter blokean.
2. Status_code 400 edo handiagoa bada, gehitu errorea etiketa.
3. Gainerako eskaerei ez gehitu etiketarik.
4. Exekutatu pipeline-a eta egiaztatu emaitza.

PISTA:

```
if [status_code] >= 400 {  
  mutate {  
    add_tag => ["errorea"]  
  }  
}
```

- Zein logek jasotzen dute errorea etiketa?
- 200 egoera-kodea duen logak etiketa jasotzen du?
- 500 kodea duenak?

Gogoan hartzekoak

-
- i* **Alderantzizko indizea** — zergatik azkarragoa SQL LIKE baino

 - ii* **text vs keyword** — term text eremuan ez du ezer aurkitzen

 - iii* **bool**: must (score) vs **filter** (cacheatua, iragazketarako)

 - iv* **Agregazioak**: terms (GROUP BY), date_histogram, percentiles (p99)

 - v* **Logstash**: grok (eremuak) + date (@timestamp ezinbestekoa)

ESKERRAK

Galderak eta eztabaida.

GAIA

2 · Elastic Stack